
Meccano Robot Control: Development Story

The story of building a GUI to control a Meccano Meccanoid G15 from a Raspberry Pi, told milestone by milestone. It's updated with every pull request; the PDF version ([docs/STORY.pdf](#)) is generated from this file.

Last updated: 1 October 2026

Current status: Software complete for a first hardware test. Not yet run against a real robot.

At a glance

Goal	Control a Meccanoid G15 from any phone or laptop, via a Raspberry Pi
Robot link	Bluetooth Low Energy, reverse-engineered protocol
Software	Python 3.11+, bleak (Bluetooth), FastAPI (web server), plain HTML/CSS/JS
Repository	https://github.com/robzarry/meccanorobotcontrol
Pull requests merged	2
Automated tests	27, all passing
Hardware tested	Not yet

1. The idea

The Meccanoid G15 is a 2-foot build-it-yourself robot from Meccano (Spin Master). It was designed to be driven by a phone app over Bluetooth, but that app is no longer maintained and Meccano never published an API. The aim of this project is to replace the app with our own control system: a Raspberry Pi sits near the robot, talks to it over Bluetooth, and serves a control panel that any phone or laptop on the home network can open in a browser.

2. Research: what the community already knew

With no official documentation, the first step was finding out what others had worked out.

- **pymecca** (github.com/iamsrp/pymecca) is a small, unofficial Python library from the community. It records the robot's message format: every command is exactly 20 bytes (18 bytes of content and a 2-byte checksum), written to one Bluetooth "characteristic" (a numbered mailbox on the device, handle `0x001f`). It knows how to move the 8 servo slots, colour the servo lights, drive the wheels, set the eye colour, switch the chest lights and trigger a wake-up animation.
- pymecca relies on [gatttool](#), a Bluetooth command-line tool that is **deprecated** on current Raspberry Pi OS, so it couldn't be used as-is.
- **A Raspberry Pi forum thread** described someone whose G15 connected but then ignored every command and dropped the connection. Their robot may use a different mailbox. This became the project's biggest open risk.

What the robot understands, as far as the community knows:

Command	First byte	What it does
Servo positions	0x08	Sets all 8 servo slots at once (0-255, centre 128)
Servo lights	0x0c	LED colour per servo: off, red, green, yellow, blue, magenta, cyan, white
Wheels	0x0d	Direction and speed for left and right wheels
Eyes	0x11	Eye colour, 8 levels each of red, green and blue
Wake	0x19	Yawn, "I'm awake", arm wiggle
Chest lights	0x1c	Four chest lights on/off

3. Key decisions

Decision	Why
Python on the Pi	Raspberry Pi OS ships with it; the community code is Python
bleak for Bluetooth	Modern and maintained, unlike the deprecated <code>gatttool</code>
A web page as the GUI	Works on any phone, tablet or laptop with nothing to install
A mock robot	Build and test everything on a Mac without the hardware
A hardware probe script first	Confirms the riskiest unknown (does our G15 accept these commands?) cheaply
Small pull requests	One reviewable change at a time, each with tests

How the pieces fit:

```
phone / laptop browser --Wi-Fi--> Raspberry Pi (Python) --Bluetooth LE--> Meccanoid
                                   protocol.py  builds the 20-byte messages
                                   transport.py   real Bluetooth or mock robot
                                   robot.py       remembers state, sends commands
                                   server.py      web panel + live connection
```

4. Milestone 1: The foundation (PR #1)

The first pull request built everything needed to talk to the robot, but no GUI yet.

- **protocol.py** turns commands into the exact 20-byte messages and refuses out-of-range values before they ever reach the robot.
- **transport.py** has two ways to reach a robot: real Bluetooth (bleak), which automatically finds the right mailbox on connect, and a mock robot that just records what it was sent.
- **robot.py** remembers the robot's current pose and lights. This matters because the robot only accepts all 8 servos in a single message, so moving one arm joint means re-sending the other seven too.

-
- **scripts/probe.py** is a three-step check to run on the Pi: **scan** to find the robot, **inspect** to list its Bluetooth mailboxes, **test** to send the wake-up animation and cycle the eye colours.

Proof: 20 automated tests. The wake-up message our code builds matches, byte for byte, the one captured in the forum thread, checksum included.

5. Milestone 2: The web control panel (PR #2)

The second pull request added the part you actually use: a control panel served by the Pi.

- **A live drawing of the robot** that mirrors its state: eye colour, arm angles, chest lights, servo lights and which way the wheels are turning.
- **A hold-to-drive pad** (forward, back, spin left, spin right) with a speed slider. Arrow keys and WASD work on a keyboard.
- **Arm sliders** for the four known joints, plus the four unidentified servo slots tucked away for experimenting.
- **Eye colour** presets and sliders, **chest light** toggles and **servo light** colours.
- **Wake up, Centre arms**, and a big red **STOP** button always on screen.
- Every open browser stays in sync, because the state lives on the Pi and is pushed to every page after each change.

Designing for safety

A rolling robot that won't stop is the worst failure, so the wheels act as a **dead-man's switch**: the page re-sends "drive" every 0.2 seconds while a button is held, and the Pi stops the wheels if it hears nothing for 0.6 seconds. That covers a released button, a switched tab, a locked phone and a dropped Wi-Fi connection. Closing the page that was driving also stops the wheels immediately.

A bug the tests caught

Two of the arm servos are mounted mirror-image, so their values are flipped (255 minus the position). The new tests showed that "centre" (128) read back as 127 on those two joints, an off-by-one that would have made the sliders jump. Fixed by defining centre per joint rather than as one raw value.

Checking it by eye

Beyond the tests, the panel was driven in a real browser against the mock robot: colours, arms and lights all updated the drawing; holding drive for 1.5 seconds kept the wheels going; releasing, switching windows and STOP all stopped them. At phone size the robot drawing pushed the drive pad off the first screen, so it was shrunk for narrow screens.

Proof: 27 automated tests, including the drive time-out and stop-on-disconnect.

6. Bumps in the road

- **Wrong folder.** Work briefly started in an unrelated project before moving to this repository. Nothing was written there.

-
- **The stacked PR that closed itself.** PR #2 was built on top of PR #1's branch. After merging #1, its branch was deleted, and GitHub *closed* #2 instead of moving it to `main`. It was recovered by restoring the branch, reopening #2 and pointing it at `main`. Nothing was lost. *Lesson:* retarget a stacked PR to `main` before deleting the branch under it.

7. Where things stand

Done: protocol library, Bluetooth and mock connections, hardware probe, web control panel with safety stop, 27 tests, all merged to `main`.

Open questions:

- Does our G15 accept commands on the same Bluetooth mailbox as pymecca's robot? The forum report suggests some don't.
- What are the real safe limits of each arm joint? The sliders currently allow the full 0-255 range, and the drawing's angles are approximate.
- What do the four unidentified servo slots control on a G15?

Next steps:

- 1 Run `probe.py scan` and `probe.py test` on the Pi with the robot.
- 2 If it works, drive it from the panel; if not, use `probe.py inspect` to find the right mailbox.
- 3 Measure joint limits and clamp the sliders to them.
- 4 Start the panel automatically when the Pi boots.
- 5 Add a GitHub check that runs the tests on every pull request.

Timeline

Date	Event
1 Oct 2026	Project started; community protocol research (pymecca, Pi forum)
1 Oct 2026	Repository set up; PR #1 opened: protocol, transports, probe script
1 Oct 2026	PR #2 opened: web control panel, safety stop, mirrored-joint fix
1 Oct 2026	PRs #1 and #2 merged to <code>main</code> (with the stacked-PR recovery)
1 Oct 2026	Development story and PDF added